SYSTEM OBSŁUGI SIECI PACZKOMATÓW • DOKUMENTACJA PROJEKTOWA

Opis projektu i zmian — grupa „Krzycha”

Podsumowanie architektury, testów procesu nadania/dostarczenia (T01–T20) oraz modułu zwrotów częściowych (R1–R8)

> Plan został w pełni zrealizowany. Wszystkie wymagania prowadzącego dotyczące **testów procesu nadania/dostarczenia** oraz **rozbudowy o obsługę zwrotów (w tym zwrotów częściowych)** zostały zaimplementowane, przetestowane i pomyślnie zweryfikowane (**70/70 testów zaliczonych**), przy zachowaniu 100% izolacji (folder `nasz/` nie został w żaden sposób zmodyfikowany).
>
> Poniżej znajduje się kompletne podsumowanie i przewodnik po projekcie, który możesz bezpośrednio wykorzystać do obrony i prezentacji przed prowadzącym.

1. O co chodzi w projekcie grupy „Krzycha”?

Projekt to system obsługi sieci paczkomatów w architekturze warstwowej **.NET 10 / C# / Entity Framework Core / MySQL / Blazor Server**:

1. `PackageDeliverySystem.Core`:
   - Rdzeń domenowy (zero zależności zewnętrznych). Zawiera encje (`Package` , `Box` , `ParcelLocker` , `Payment` , `User` , `Courier` , `Operator`), enumy (`PackageStatus` , `BoxSize` , `PaymentStatus`) oraz interfejsy serwisów i repozytoriów.
   - **Enkapsulacja reguł biznesowych**: Status paczki nie jest zmieniany bezpośrednio, lecz przez dedykowane metody domenowe (`Accept()` , `Prepare()` , `Ship()` , `Deliver()` , `Collect()` , `Cancel()` , `Reject()` , `Expire()`), które sprawdzają niezmienniki biznesowe (np. `Accept()` wymaga wcześniejszego potwierdzenia wpłaty oraz fizycznego włożenia paczki do skrytki).
2. `PackageDeliverySystem.Infrastructure`:
   - Dostęp do bazy danych przez EF Core (`DeliveryDbContext`), repozytoria (`PackageRepository` , `BoxRepository`), serwisy domenowe (`PackageOrderService` , `PackageIntakeService` , `PackageAcceptanceService` , `PackageFulfillmentService` , `RatingService`) oraz integracje (symulator bramki płatności `FakePaymentGateway` , powiadomienia e-mail przez MailKit/Hangfire).
3. `PackageDeliverySystem.Tests`:
   - Testy jednostkowe oparte o **xUnit** oraz **Moq** — działające w pamięci RAM, bez potrzeby uruchamiania kontenerów Dockera czy bazy MySQL.
4. **Cykl życia przesyłki (Flow)**:

   `Ordered` → *[opłacenie + fizyczny depozyt]* → `Accepted` → `Preparing` → `Shipped` → `Delivered (kod)` → *[kod odbioru]* → `Collected`

2. Zadanie 1: Testy procesu nadania i dostarczenia (T1–T20)

W pliku `PackageLifecycleTests.cs` zaimplementowano **20 scenariuszy testowych**, ściśle podzielonych na 3 części wymagane przez prowadzącego:

1. **Arrange (Przygotowanie)** — konfiguracja encji, mocków repozytoriów i serwisów,
2. **Act (Właściwy test)** — wywołanie konkretnych metod z konkretnymi parametrami,
3. **Assert (Weryfikacja rezultatu)** — sprawdzenie zwracanych obiektów, statusów i weryfikacja interakcji z mockami (`Verify`).

Przegląd scenariuszy testowych

Kod	Scenariusz	Wywoływane metody i parametry	Weryfikowany rezultat
T01	**Pełny cykl dostawy (Happy Path)**	`ConfirmDeposit()`, `Payment.Complete()`, `Accept()`, `Prepare()`, `Ship()`, `Deliver()`, `Collect(PickupCode)`	Status `Collected`, wygenerowany 6-cyfrowy `PickupCode`, komplet timestampów.
T02	**Anulowanie na etapie zamówienia (Ordered)**	`package.Cancel()`	Status zmienia się z `Ordered` na `Cancelled`.
T03	**Anulowanie po akceptacji (Accepted)**	`package.Accept()`, następnie `package.Cancel()`	Status `Cancelled` (paczka jeszcze nie została przekazana kurierowi).
T04	**Blokada anulowania po wysyłce**	`package.Ship()`, próba `package.Cancel()`	Rzuca `InvalidOperationException("Cannot cancel package in status Shipped")`.
T05	**Błąd płatności i ponowienie**	`RequestPaymentAsync()`, `gateway.Fail()`, `ConfirmPaymentAsync()`, ponowne `RequestPaymentAsync()`	Status płatności `Failed`, możliwość ponowienia i pomyślnego przejścia w `Completed`.
T06	**Kurier nie znajduje paczki (pusta skrytka)**	`intakeService.ReportMissingDepositAsync(packageId, courierId)`	Status przesyłki `Cancelled`, wysłana notyfikacja systemowa obciążająca winą nadawcę.
T07	**Przeterminowanie depozytu (missed deposit)**	`package.DepositDeadline = UtcNow - 1h`, `intakeService.ProcessMissedDepositsAsync()`	Automatyczne anulowanie (`Cancelled`) przez zadanie cykliczne.
T08	**Zgubienie paczki w transporcie (timeout kuriera)**	Status `Shipped`, upłynięcie 48h okna tranzytowego, `package.Expire()`	Status przesyłki `Expired`.

Kod	Scenariusz	Wywoływane metody i parametry	Weryfikowany rezultat
T09	**Kurier próbuje doręczyć bez wysyłki**	Status `Accepted` (lub `Preparing`), próba `package.Deliver()`	Rzuca `InvalidOperationException("Cannot deliver package in status Accepted")`.
T10	**Klient podaje błędny kod odbioru**	Status `Delivered`, `package.Collect("000000")`	Rzuca `InvalidOperationException("Invalid pickup code")`, status pozostaje `Delivered`.
T11	**Brak odbioru przez klienta w 48h**	Status `Delivered`, `package.ExpiredAt = UtcNow - 1h`, `package.Expire()`	Status przesyłki `Expired`.
T12	**Nieuprawniony użytkownik próbuje odebrać**	`fulfillmentService.CollectPackageAsync(packageId, wrongUserId, correctCode)`	Rzuca `InvalidOperationException("Only this package's recipient can collect it")`.
T13	**Odrzucenie przez operatora z Ordered**	`acceptanceService.RejectPackageAsync(packageId, operatorId)`	Status `Rejected`, notyfikacja do systemu.
T14	**Odrzucenie przez operatora z Accepted**	`package.Accept()`, `acceptanceService.RejectPackageAsync(packageId, operatorId)`	Status `Rejected`.
T15	**Blokada odrzucenia po wysyłce**	Status `Shipped`, próba `package.Reject()`	Rzuca `InvalidOperationException`.
T16	**Próba akceptacji bez depozytu**	Płatność `Completed`, ale `DepositConfirmedAt = null`, `package.Accept()`	Rzuca `InvalidOperationException("Cannot accept a package that was never deposited...")`.
T17	**Próba akceptacji bez opłacenia**	`ConfirmDeposit()`, ale brak płatności, `package.Accept()`	Rzuca `InvalidOperationException("Cannot accept a package that has not been paid for")`.
T18	**Idempotencja potwierdzenia płatności**	Wywołanie `ConfirmPaymentAsync()` na już opłaconej paczce	Ciche zakończenie sukcesem (no-op), brak podwójnych powiadomień.
T19	**Walidacja ocen (Rating)**	`rateService.RateCourierAsync(id, recipientId, score: 6)` oraz `score: 0`	Rzuca `ArgumentOutOfRangeException("Score must be between 1 and 5")`.

Kod	Scenariusz	Wywoływane metody i parametry	Weryfikowany rezultat
T20	**Przypisanie kuriera w błędnym statusie**	Status `Ordered` (lub `Delivered`), próba `package.AssignCourier(courier)`	Rzuca `InvalidOperationException("Cannot assign courier to package in status Ordered")`.

3. Zadanie 2: Rozbudowa funkcjonalności — Obsługa Zwrotów (w tym częściowych)

3.1. Jak rozwiązano problem zwrotów częściowych?

W pierwotnej architekturze jedno zamówienie reprezentowane było przez pojedynczą encję `Package`. Aby umożliwić **częściowy zwrot** (np. zwrot jednego z kilku produktów zamówionych w ramach jednego koszyka), wprowadzono:

- **`Package.OrderGroupId`**: Identyfikator logicznego zamówienia nadrzędnego łączący wiele przesyłek/produktów.
- Dzięki temu klient może w ramach jednego zamówienia (np. `OrderGroupId = 500`) posiadać 3 paczki, z czego **tylko jedna** wchodzi w cykl zwrotu, a pozostałe pozostają odebrane (`Collected`).

3.2. Nowe encje i rozszerzenia modelu (`PackageDeliverySystem.Core`)

1. **Nowa encja `Return.cs`**:
   - Pola: `Id`, `PackageId`, `Status`, `Reason`, `RequestedAt`, `ApprovedAt`, `CompletedAt`, `ReturnBoxId`, `ReturnDestinationBoxId`.
   - Metody domenowe: `Approve()`, `Reject()`, `Ship()`, `Complete()`, `Cancel()`.
2. **Nowy enum `ReturnStatus.cs`**:
   - Wartości: `Requested`, `Approved`, `Rejected`, `Shipped`, `Completed`, `Cancelled`.
3. **Rozszerzenie `PackageStatus.cs`**:
   - Dodano: `ReturnRequested`, `ReturnInProgress`, `Returned`.
4. **Rozszerzenie płatności (`Payment.cs` / `PaymentStatus.cs`)**:
   - Dodano status `PaymentStatus.Refunded` oraz metodę domenową `payment.Refund()`.
5. **Kontrakty serwisów** (`IReturnRepository.cs`, `IReturnService.cs`):

```
Task<Return> RequestReturnAsync(int packageId, int customerId, string reason);
Task ApproveReturnAsync(int returnId, int operatorId, int returnBoxId, int
returnDestBoxId);
Task RejectReturnAsync(int returnId, int operatorId);
Task ShipReturnAsync(int returnId, int courierId);
Task CompleteReturnAsync(int returnId);
Task<IEnumerable<Package>> GetReturnablePackagesAsync(int orderGroupId, int
customerId);
```

3.3. Implementacja w warstwie infrastruktury (`PackageDeliverySystem.Infrastructure`)

1. `ReturnRepository`: Asynchroniczne operacje bazy danych z EF Core, ładowanie relacji `Include(r => r.Package).ThenInclude(p => p.Payment)`.
2. `ReturnService`: Pełna orkiestracja procesu zwrotu:
   - Weryfikacja, że zwracać może wyłącznie odbiorca (`RecipientId == customerId`).
   - Weryfikacja, że paczka ma status `Collected`.
   - Blokada wielokrotnych zwrotów tej samej paczki.
   - Obsługa zwrotu częściowego przez `GetReturnablePackagesAsync`.
   - Zmiana statusu płatności na `Refunded` po ukończeniu zwrotu (`CompleteReturnAsync`).
3. `DeliveryDbContext`: Dodanie `DbSet<Return>` oraz konfiguracja relacji 1:1 i kluczy obcych.

4. Testy modułu zwrotów (R1–R8)

W pliku `ReturnTests.cs` zaimplementowano **41 testów jednostkowych** weryfikujących wszystkie wymagane scenariusze:

- **R1 (Poprawne zgłoszenie zwrotu)**: Zgłoszenie zwrotu dla odebranej paczki tworzy rekord ze statusem `Requested` i przestawia paczkę w `ReturnRequested`.
- **R2 (Blokada przed odbiorem)**: Próba zwrotu paczki w statusie `Ordered`, `Accepted`, `Preparing`, `Shipped` rzuca `InvalidOperationException`.
- **R3 (Zatwierdzenie przez operatora)**: `ApproveReturnAsync` alokuje skrzynki, ustawia status zwrotu `Approved` oraz status paczki `ReturnInProgress`.
- **R4 (Odrzucenie przez operatora)**: `RejectReturnAsync` zmienia status zwrotu na `Rejected` i przywraca paczce status `Collected`.
- **R5 (Obsługa przez kuriera)**: `ShipReturnAsync` (status `Shipped`) → `CompleteReturnAsync` (status `Completed` i paczka `Returned`).
- **R6 (Zwrot częściowy)**: Utworzenie 3 paczek z tym samym `OrderGroupId`. Zwrot paczki #1 zmienia status tylko paczki #1, podczas gdy paczki #2 i #3 pozostają w nienaruszonym statusie `Collected`. Metoda `GetReturnablePackagesAsync` poprawnie filtruje pozostałe pozycje.
- **R7 (Blokada podwójnego zwrotu)**: Ponowna próba zgłoszenia zwrotu paczki, która ma już aktywny zwrot, jest blokowana wyjątkiem.
- **R8 (Automatyczna refundacja)**: Zakończenie zwrotu (`CompleteReturnAsync`) automatycznie wywołuje `payment.Refund()`, ustawiając status płatności na `Refunded`.

5. Jak uruchomić i sprawdzić projekt?

Przejdź do katalogu projektu:

```
cd "c:\Users\filip\OneDrive\Desktop\STUDIA\SEM 3\inzynieria\package-management\krzycha"
```

1. **Uruchomienie wszystkich testów** (wynik: **70/70 testów przechodzi — 100% pass**):

```
dotnet test
```

2. **Uruchomienie wyłącznie testów cyklu życia (T1–T20)**:

```
dotnet test --filter "FullyQualifiedName~PackageLifecycleTests"
```

3. **Uruchomienie wyłącznie testów zwrotów (R1–R8)**:

```
dotnet test --filter "FullyQualifiedName~ReturnTests"
```

4. **Kompilacja całego rozwiązania** (wynik: **0 błędów, 0 ostrzeżeń**):

```
dotnet build
```

6. Wskazówki na obronę projektu (Jak uzyskać 100%)

Gdy prowadzący zapyta Cię o poszczególne elementy:

1. **O podział testu na 3 części:**

> *„Każdy nasz test w `PackageLifecycleTests` i `ReturnTests` posiada jawne sekcje `// Arrange`, `// Act`, `// Assert`. W sekcji Arrange konfigurujemy stan systemu przy użyciu mocków `Moq` (dzięki czemu testy wykonują się w ułamku sekundy bez konieczności stawiania MySQL w Dockerze), w sekcji Act wywołujemy konkretną metodę biznesową z parametrami, a w sekcji Assert sprawdzamy asercjami `Assert.Equal` / `Assert.Throws` stan encji oraz metodą `Verify` potwierdzamy wywołania repozytoriów i notyfikacji.”*

2. **O zwroty częściowe:**

> *„Paczki posiadają pole `OrderGroupId`. W przypadku zamówienia wielu produktów każdy z nich staje się osobną przesyłką w ramach tej samej grupy zamówienia. Klient może zwrócić tylko jedną wybraną paczkę – jej status przechodzi w `ReturnRequested`, a pozostałe paczki z tego zamówienia pozostają w statusie `Collected`. Zapewnia to pełną niezależność cyklu życia poszczególnych pozycji.”*

3. **O integralność domenową:**

> *„Wszystkie operacje zmiany statusu są chronione wewnątrz encji `Package` i `Return`. Zewnętrzne serwisy nie mogą dowolnie nadpisywać statusów z pominięciem reguł walidacji, co zapobiega niespójności danych.”*